



BÁO CÁO ĐỒ ÁN

Môn học: Toán cho Khoa học máy tính (CS115)

CHIẾN LƯỢC TIẾN HÓA TỰ NHIÊN (Natural Evolution Strategies - xNES)

Nhóm thực hiện: Nhóm 11

Thành viên: 1. Đinh Mạnh Hùng - 24520014
2. Lê Thiên Quân - 24520027

TP. Hồ Chí Minh, January 5, 2026

Contents

1	Giới thiệu	2
2	Cơ sở Lý thuyết và Thuật toán	2
2.1	Canonical Evolution Strategies (Canonical ES)	2
2.2	Exponential NES (xNES)	2
3	Phân tích Thực nghiệm từ File Demo	3
3.1	Thiết lập bài toán và Hàm Utilities	3
3.2	Phân tích lỗi sập hệ thống (Canonical)	3
3.3	Giải pháp ổn định từ xNES	4
4	Kết luận	4

1 Giới thiệu

Trong khuôn khổ môn học CS112, nhóm 11 thực hiện nghiên cứu và so sánh thực nghiệm về các giải thuật tối ưu hóa hộp đen. Báo cáo này dựa trên mã nguồn demo `CS112_Natural_Evolution_Strategies-demo.ipynb` để minh họa sự hạn chế của phương pháp Gradient truyền thống (Canonical) và ưu điểm của biến thể xNES.

2 Cơ sở Lý thuyết và Thuật toán

2.1 Canonical Evolution Strategies (Canonical ES)

Phương pháp này cập nhật tham số phân phối (Gaussian) dựa trên Gradient của hàm log-likelihood.

- **Cơ chế:** Sử dụng phép cập nhật cộng tính (additive update) cho cả μ và σ .
- **Vấn đề:** Gradient thường (Vanilla Gradient) không nhận thức được cấu trúc hình học của không gian xác suất. Khi cập nhật σ , nếu bước nhảy quá lớn hoặc hướng gradient âm, σ có thể bị kéo về 0 hoặc giá trị âm rất nhanh.

Thuật toán 1 Canonical ES (Vanilla Gradient - Dễ gây lỗi)

Input: Hàm f , Khởi tạo μ, σ , Hệ số học α

```

1:  $t \leftarrow 0$ 
2: while chưa hội tụ do
3:   Sinh mẫu  $z_i \sim \mathcal{N}(0, I)$ , tính  $x_i = \mu + \sigma z_i$ 
4:   Tính Utilities  $u_i$  dựa trên  $f(x_i)$ 
5:    $\nabla_\mu \approx \sum u_i z_i$ ;  $\nabla_\sigma \approx \sum u_i (z_i^2 - 1)$ 
6:    $\mu \leftarrow \mu + \alpha_\mu \cdot \sigma \cdot \nabla_\mu$ 
7:    $\sigma \leftarrow \sigma + \alpha_\sigma \cdot \nabla_\sigma$  ▷ Cập nhật cộng: Rủi ro cao!
8: end while
```

2.2 Exponential NES (xNES)

xNES giải quyết vấn đề của Canonical bằng cách sử dụng **Natural Gradient** và phép cập nhật mũ (exponential update).

- **Cơ chế:** Cập nhật trên không gian logarit của σ .
- **Ưu điểm:** Đảm bảo σ luôn dương và tỷ lệ thay đổi (relative change) là hằng số, giúp thuật toán ổn định.

Thuật toán 2 xNES (Exponential Update - Ổn định)**Input:** Hàm f , Khởi tạo μ, σ , Hệ số học η

```

1:  $t \leftarrow 0$ 
2: while chưa hội tụ do
3:   Sinh mẫu  $z_i \sim \mathcal{N}(0, I)$ , tính  $x_i = \mu + \sigma z_i$ 
4:   Tính Utilities  $u_i$ 
5:    $G_\mu = \sum u_i z_i$ ;  $G_\sigma = \sum u_i (z_i^2 - 1)$ 
6:    $\mu \leftarrow \mu + \eta_\mu \cdot \sigma \cdot G_\mu$ 
7:    $\sigma \leftarrow \sigma \cdot \exp(\eta_\sigma \cdot G_\sigma / 2)$  ▷ Cập nhật nhân: Luôn an toàn
8: end while

```

3 Phân tích Thực nghiệm từ File Demo

3.1 Thiết lập bài toán và Hàm Utilities

Dựa trên file notebook CS112...demo.ipynb, nhóm thiết lập các tham số: $\mu_{start} = 3.0, \sigma_{start} = 1.0$. Trước khi đi vào vòng lặp tối ưu, một thành phần quan trọng là hàm tính Utilities (Fitness Shaping) được sử dụng để chuẩn hóa giá trị thích nghi:

```

1 # Chuyển đổi giá trị fitness thành rank-based utilities
2 def compute_utilities(fitness):
3     n = len(fitness)
4     ranks = np.zeros(n)
5     ranks[np.argsort(fitness)] = np.arange(n)
6     # Sử dụng hàm log để lấy trong số
7     utilities = np.maximum(0, np.log(n/2 + 1) - np.log(n - ranks))
8     utilities = utilities / np.sum(utilities) - 1/n
9     return utilities

```

Listing 1: Hàm tính Utilities từ code demo

3.2 Phân tích lỗi sập hệ thống (Canonical)

Trong thực nghiệm, phương pháp Canonical sử dụng phép cập nhật cộng tính. Đoạn code dưới đây trích từ file thực nghiệm cho thấy nguyên nhân gây ra lỗi:

```

1 # ... (Trong vòng lặp)
2 # Tính Gradient ước lượng
3 grad_sigma_can = np.dot(utilities, (z_sorted_can**2 - 1))
4
5 # CẬP NHẬT THAM SỐ (Canonical - Additive Update)
6 # Đây là bước gây lỗi nếu grad_sigma âm lớn
7 sigma_can = sigma_can + lr_sigma_can * grad_sigma_can
8
9 # Safety brake: Kiểm tra điều kiện dừng
10 if sigma_can < 1e-10:
11     print("!!! HE THONG SAP VI SIGMA QUA NHỎ !!!")
12     break

```

Listing 2: Đoạn code gây lỗi System Crash trong Canonical ES

Kết quả chạy thực tế cho thấy hệ thống dừng đột ngột với thông báo:

```

Start: Mu=3.0, Sigma=1.0
...
!!! HE THONG SAP VI SIGMA QUA NHO !!!

```

Nguyên nhân: Do sử dụng Gradient thường, thuật toán "quá tự tin" đẩy σ về giá trị âm cực lớn. Phép cộng $\sigma + \Delta$ khiến σ chạm ngưỡng 10^{-10} trước khi μ kịp di chuyển về đích. Đây là hiện tượng ****Premature Convergence**** (Sụp đổ phương sai).

3.3 Giải pháp ổn định từ xNES

Ngược lại, xNES sử dụng phép cập nhật hàm mũ, đảm bảo tính ổn định số học:

```

1 # ... (Trong vòng lặp xNES)
2 grad_sigma_nes = np.dot(utilities, (z_sorted_nes**2 - 1))
3
4 # CAP NHAT THAM SO (xNES - Exponential Update)
5 # Phep nhan voi exp() dam bao sigma luon duong
6 sigma_nes = sigma_nes * np.exp(lr_sigma_nes / 2 * grad_sigma_nes)
7
8 # Khong can safety brake vi sigma khong bao gio < 0

```

Listing 3: Đoạn code cập nhật ổn định của xNES

Kết quả thực nghiệm cho thấy xNES hội tụ mượt mà về đích (0.0) mà không gặp bất kỳ lỗi nào.

Figure 1: So sánh đường đi hội tụ: Canonical bị đứt đoạn do lỗi, xNES hội tụ thành công.

4 Kết luận

Qua bài tập lớn và file demo thực nghiệm, nhóm 11 kết luận:

1. ****Không nên dùng Gradient thường (Canonical)**** cho các tham số tỉ lệ (scale parameters) như độ lệch chuẩn σ vì rủi ro sập hệ thống cao.
2. ****xNES là lựa chọn tối ưu**** nhờ tính chất bất biến (invariance) và sự ổn định số học thông qua phép cập nhật mũ.

References

- [1] Source code thực nghiệm Nhóm 11, *CS112_Natural_Evolution_Strategies-demo.ipynb*.
- [2] Wierstra, D., et al. (2014). *Natural Evolution Strategies*. Journal of Machine Learning Research.